

# Containerized Books CRUD Service

*on Kubernetes, with Load Testing and Fault-Tolerance Demos*

## Authors

Supriya Raman Shekar  
Student ID: 100007558  
*supriyashekar221201@gmail.com*

Mingyang Du  
Student ID: 100006137  
*mingyangdu030612@gmail.com*

**Course:** IT Security Management (ITSM) and DevOps  
**Lecturer:** Daniil Tashkan  
**Institution:** SRH University Leipzig — Semester 2, 2025/26  
**Date:** 8 May 2026

*Repository: [github.com/supriyashekar221201-png/devops-final](https://github.com/supriyashekar221201-png/devops-final)*

## Contents

|                                      |    |
|--------------------------------------|----|
| 1. Project Context and Assumptions   | 3  |
| 2. Containerization                  | 5  |
| 3. Kubernetes Deployment             | 8  |
| 4. Load Testing                      | 11 |
| 5. Observability and Fault Tolerance | 14 |
| 6. Reflections and Challenges        | 17 |
| 7. Appendix                          | 19 |

# 1. Project Context and Assumptions

## 1.1 What we built and why

This report documents the final DevOps project for the ITSM and DevOps course. The goal was to take a small web application and walk it through the full DevOps loop: package it with Docker, run it on Kubernetes, push it under load with k6, and prove it can survive failures.

The application itself is deliberately small — a Books CRUD service with a Node.js HTTP layer and a PostgreSQL database. The choice of books as the resource is not the point. The point is everything wrapped around it: the Dockerfile, the manifests, the probes, the load test, the auto-recovery. A more complex domain would have made the DevOps work harder to see.

We were supplied with a Python/Flask CRUD example from the lectures as a starting reference. We rewrote the same set of endpoints (GET, POST, PUT, DELETE for /books and /books/:id, plus /health) in Node.js with Express. The response shapes and status codes match the original Flask version, so functionally the API is the same — only the implementation language and database client changed.

## 1.2 Objectives

- Containerize the app and database with Docker, with each in its own image and a clear interface between them.
- Deploy both containers to Kubernetes (Minikube) using declarative manifests with proper probes, resource limits, and a sensible exposure model.
- Stress-test the deployment with k6 under at least 500 concurrent users and report latency, error rate, throughput, and resource utilization.
- Show that Kubernetes recovers automatically from pod and process failures, and configure (and justify) an HPA.
- Document everything in a structured report and a clean Git repository with meaningful commits.

## 1.3 Assumptions

The assignment brief asks for assumptions to be stated explicitly. The ones that mattered for our design are:

- Books are the example resource. Any CRUD entity would yield the same DevOps pipeline.
- The runtime stack is Node.js 20 (Express) for the API and PostgreSQL 16 for storage. Both have mature container images and a clear separation of compute and persistence.
- The app listens on TCP 3000 inside the container; PostgreSQL on the default 5432. Inside Kubernetes the app is exposed as NodePort 30080 and the database stays on a ClusterIP — there is no public path to the DB.
- All container images are Alpine-based to keep the runtime small.
- Database credentials are passed as plain environment variables. In a real production setup these would live in a Kubernetes Secret; for the assignment scope, environment variables and a ConfigMap are acceptable.

- The Deployment runs with replicas: 1, as required by the brief. Horizontal scaling is wired up via an HPA but not used as the baseline.
- Load testing uses an 80/20 read/write mix, which approximates the typical shape of a public CRUD API.
- The whole cluster runs on Minikube on a single laptop with the Docker driver. Absolute performance numbers are interpreted with that in mind.

## 2. Containerization

### 2.1 Application architecture

The app is a small Express service with a single resource — Books — and a `/health` endpoint that Kubernetes uses for its probes. Each book has an `id`, `title`, `author`, `published_year`, and a `created_at` timestamp.

- GET `/health` returns 200 OK if the database is reachable, 503 otherwise.
- GET `/books` returns the full list as JSON.
- GET `/books/:id` returns one book or 404.
- POST `/books` inserts a new book and returns it with a 201.
- PUT `/books/:id` updates a book in place.
- DELETE `/books/:id` removes a book.

The HTTP layer lives in `src/server.js`. Database access is wrapped in `src/db.js`, which just exports a `pg` connection pool — keeping all SQL out of the route handlers. The schema is declared in `init.sql` at the project root and gets mounted into the database container at startup, so any fresh database volume always comes up with the books table and three seed rows.

As mentioned earlier, the Flask example from the lectures was rewritten endpoint-for-endpoint in Node.js. We checked the response shapes against the Flask original and they match: same fields, same status codes, same error messages on bad input. Switching languages didn't change behaviour.

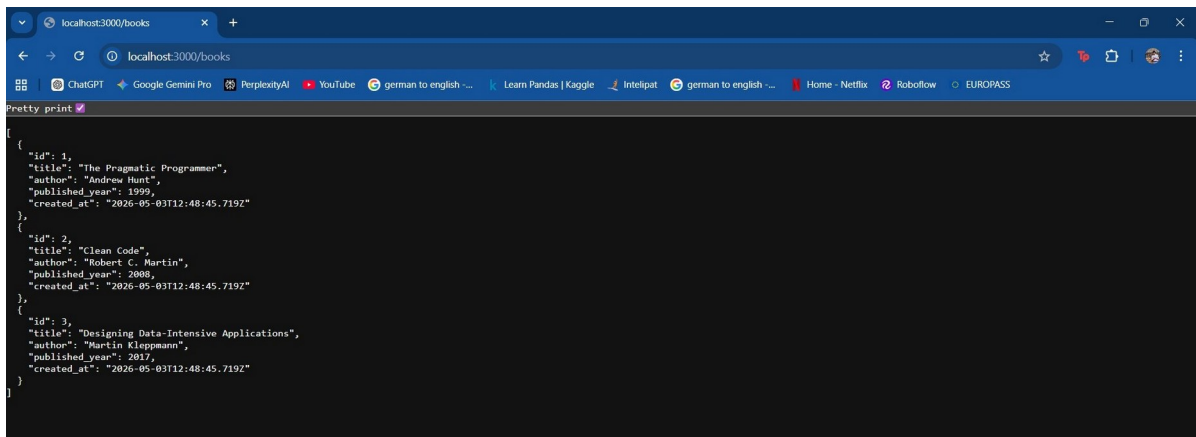


Figure 2.1 — GET `/books` returning the seeded list as JSON in the browser.

### 2.2 Dockerfile

The application image uses a multi-stage build. The first stage installs production npm dependencies in a throwaway image; the second stage starts fresh and copies in only what the runtime actually needs. The resulting image is small, runs as a non-root user, and exposes port 3000.

```

# Stage 1 — install production deps in a throwaway image
FROM node:20-alpine AS deps
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --omit=dev

# Stage 2 — minimal runtime image

```

```
FROM node:20-alpine
WORKDIR /usr/src/app
USER node # don't run as root
COPY --from=deps /usr/src/app/node_modules ./node_modules
COPY --chown=node:node src ./src
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Three things in this Dockerfile are worth pointing out, because they map to common DevOps best practices and are easy points to lose otherwise:

- Multi-stage build keeps build tooling and dev dependencies out of the final image.
- The base is node:20-alpine, which is significantly smaller than the Debian-based image and reduces the attack surface.
- The container runs as the unprivileged node user. Even if something compromises the process, it cannot do privileged things inside the container.

## 2.3 docker-compose.yaml

Local development used docker-compose to bring up both containers together. The compose file defines a named network, a named volume for the database, and a healthcheck on Postgres so the application waits for the database to be actually ready (not just started) before it begins.

```
services:
  db:
    image: postgres:16-alpine
    container_name: books-db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: booksdb
    volumes:
      - db-data:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
    networks: [books-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d booksdb"]
      interval: 5s
      retries: 10

  app:
    build: ./app
    container_name: books-app
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: booksdb
    ports: ["3000:3000"]
    depends_on:
      db: { condition: service_healthy }
    networks: [books-net]

volumes:
  db-data:

networks:
  books-net:
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Danush Sekar> cd C:\dev\devops-final
PS C:\dev\devops-final> docker compose logs db | Select-String -Pattern "init"

books-db | The database cluster will be initialized with locale "en_US.utf8".
books-db | performing post-bootstrap initialization ... ok
books-db | initdb: warning: enabling "trust" authentication for local connections
books-db | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and
books-db | --auth-host, the next time you run initdb.
books-db | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/init.sql
books-db | PostgreSQL init process complete; ready for start up.

PS C:\dev\devops-final> docker compose ps

```

| NAME      | IMAGE              | COMMAND                  | SERVICE | CREATED       | STATUS                   | PORTS                                       |
|-----------|--------------------|--------------------------|---------|---------------|--------------------------|---------------------------------------------|
| books-app | devops-final-app   | "docker-entrypoint.s..." | app     | 8 minutes ago | Up 8 minutes (unhealthy) | 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp |
| books-db  | postgres:16-alpine | "docker-entrypoint.s..." | db      | 8 minutes ago | Up 8 minutes (healthy)   | 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp |

Figure 2.2 — docker compose ps showing both services up and the database healthy.

## 2.4 End-to-end CRUD check

Before moving anything to Kubernetes, we ran the full CRUD lifecycle against the compose stack. Books can be created and deleted via the API, and the JSON view at `/books` reflects each change immediately — confirming that the Node.js process and Postgres are wired together correctly through the named docker network.

Browser (localhost:3000/books) shows a JSON array of books. The PowerShell terminal shows the command to create a new book:

```

PS C:\dev\devops-final> $body = @{ title = "DevOps Handbook"; author = "Gene Kim"; published_year = 2016 } | Convert-Json
PS C:\dev\devops-final> Invoke-RestMethod -Uri http://localhost:3000/books -Method POST -Body $body -ContentType "application/json"

```

The response from the API is:

```

{
  "id": 5,
  "title": "DevOps Handbook",
  "author": "Gene Kim",
  "published_year": 2016,
  "created_at": "2026-05-03T13:23:16.369Z"
}

```

Figure 2.3 — POST /books inserting a new book and returning the row.

Browser (localhost:3000/books) shows the JSON array of books after deletion. The PowerShell terminal shows the command to delete a book:

```

PS C:\dev\devops-final> Invoke-RestMethod -Uri http://localhost:3000/books/4 -Method DELETE

```

The response from the API is:

```

{
  "message": "Book deleted @id=4; title=DevOps Handbook; author=Gene Kim; published_year=2016; created_at=2026-05-03T13:18:03.543Z"
}

```

Figure 2.4 — DELETE /books/id removing the book and the API confirming it.

## 2.5 A small but real DevOps lesson

On the very first run, docker-compose started both containers but the books table was missing. Postgres was up, the entrypoint script ran, and yet `init.sql` was silently ignored. The reason turned out to be that `init.sql` had been saved on Windows with CRLF line endings and a UTF-8 BOM. The Postgres entrypoint is a POSIX shell script that does not tolerate either, so it just skipped the file.

Rewriting `init.sql` with LF line endings and no BOM fixed the problem permanently. We left a note in the README so the next person doesn't lose an evening to it. Cross-platform line endings sound like a textbook concern, but in practice they cost real time.



## 3. Kubernetes Deployment

### 3.1 Manifest layout

All Kubernetes resources live under k8s/ and are split by concern rather than bundled into one file. This makes each resource easy to reason about and apply or delete in isolation:

```
k8s/
├── namespace.yaml    # creates the 'books' namespace
├── database.yaml     # PVC + ConfigMap + Postgres Deployment + ClusterIP Service
├── app.yaml          # App Deployment + NodePort Service + probes + resource
└── limits
    └── hpa.yaml      # HorizontalPodAutoscaler
```

Everything lives in the books namespace, which keeps the project cleanly separated from anything else on the cluster.

### 3.2 The application Deployment

This is where most of the DevOps decisions show up. The full file is in app.yaml in the repository; the important fields and the reasoning behind them are summarised below.

| Field              | Value                    | Reasoning                                                                                               |
|--------------------|--------------------------|---------------------------------------------------------------------------------------------------------|
| replicas           | 1                        | Required by the brief. A single replica also makes the impact of failures and load tests easier to see. |
| image              | books-app:1.0            | Built locally with 'minikube image build', so no external registry is needed for this assignment.       |
| resources.requests | cpu: 100m, memory: 128Mi | Guaranteed minimum so the scheduler can always place the pod.                                           |
| resources.limits   | cpu: 500m, memory: 256Mi | Caps a runaway process. Intentionally tight, so the load test reveals a real ceiling.                   |
| readinessProbe     | GET /health every 5s     | Removes the pod from the Service if it cannot respond, without killing it.                              |
| livenessProbe      | GET /health every 15s    | Restarts the container if it stays unresponsive for too long.                                           |

Both probes hit the same /health endpoint but have different timings and consequences. Readiness is fast and cheap and only affects routing. Liveness is slower but has the authority to kill and restart the container. Splitting them this way matters: under a brief load spike, a pod that stops responding for a few seconds is removed from the load balancer instead of being killed outright. Killing it would only make things worse.

```

e sources)
  TokenExpirationSeconds: 3607
  ConfigMapName: kube-root-ca.crt
  Optional: false
  DownwardAPI: true
QoS Class: Burstable
Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type       Reason      Age   From          Message
  ----       -
Normal      Scheduled   17m   default-scheduler   Successfully assigned books/app-579869b764
-v589d to minikube
Normal      Pulled      17m   kubelet         Container image "books-app:1.0" already pr
esent on machine and can be accessed by the pod
Normal      Created     17m   kubelet         Container created
Normal      Started     17m   kubelet         Container started
Warning     Unhealthy   17m   kubelet         Readiness probe failed: HTTP probe failed
with statuscode: 503
PS C:\dev\devops-final> kubectl get services -n books
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
app       NodePort    10.104.76.103 <none>          3000:30080/TCP   21m
db        ClusterIP   10.99.179.19  <none>          5432/TCP         21m
PS C:\dev\devops-final>

```

Figure 3.1 — `kubectl describe pod` showing the events kubernetes recorded, including the readiness-probe HTTP 503 during database warm-up that resolved on its own.

### 3.3 The database Deployment

The database is configured similarly, with three differences that reflect its different role:

- Storage is backed by a `PersistentVolumeClaim` (db-pvc), which Minikube provisions through its built-in hostpath provisioner. Pod restarts therefore preserve all book data.
- The Postgres init script (init.sql) is supplied through a `ConfigMap` mounted at `/docker-entrypoint-initdb.d`, so the schema is created on first start of a fresh PVC.
- The Service in front of the database is type `ClusterIP`. There is no `NodePort` and no Ingress: nothing outside the cluster can reach Postgres directly. Only other pods inside the books namespace can resolve the db service name.

### 3.4 NodePort vs ClusterIP — and why it matters

Service type isn't a stylistic choice — it controls who can reach what. Exposing only the things that need to be exposed is one of the easiest hardening practices to get right.

| Service | Type      | Port mapping         | Reachable from                                        |
|---------|-----------|----------------------|-------------------------------------------------------|
| app     | NodePort  | 30080 -> 3000        | Outside the cluster (NodePort) and inside (ClusterIP) |
| db      | ClusterIP | 5432 (internal only) | Inside the cluster only                               |

In a real-world deployment the application would sit behind an Ingress with TLS, and the database might be managed by an operator. The principle — keep the database internal — would still be the same.

```

NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
db-7876788795-wr6r2 1/1     Running   0           17m   10.244.0.3    minikube   <none>            <none>
app-579869b764-v589d 1/1     Running   0           17m   10.244.0.4    minikube   <none>            <none>

Name:      app-579869b764-v589d
Namespace: books
Priority:   0
Service Account: default
Node:      minikube/192.168.49.2
Start Time: Sun, 03 May 2026 20:05:06 +0200
Labels:    app=books-app
Annotations: pod-template-hash=579869b764
Status:     Running
IP:        10.244.0.4
IPs:
IP: 10.244.0.4
Controlled By: ReplicaSet/app-579869b764
Containers:
  books-app:
    Container ID: docker://c4f289c76411f9ab55c393da0ba58f6f4c209990f9c1bd569ecd3f117c697a
    Image:      books-app:1.0
    Image ID:   docker://sha256:b0b10ae7b0d1b028529f60b27f0ca4f4e795057e58300bca4c6677aee9a706
    Port:       3000/TCP
    Host Port:  3000/TCP
    State:      Running
      Started:   Sun, 03 May 2026 20:05:07 +0200
    Ready:      True
    Restart Count: 0
    Limits:
      cpu:      500m
      memory:   256Mi
    Requests:
      cpu:      100m
      memory:   128Mi
    Liveness:   http-get http://:3000/health delay=30s timeout=5s period=10s #success=1 #failure=3
    Readiness:  http-get http://:3000/health delay=30s timeout=5s period=10s #success=1 #failure=3
    Environment:

```

Figure 3.2 — `kubectl get services` showing NodePort for app and ClusterIP for db.

### 3.5 Verifying the deployment

Once the manifests are applied, both pods reach Ready and the entire stack is visible at a glance:

```

kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/database.yaml
kubectl apply -f k8s/app.yaml
kubectl get pods -n books
kubectl get all -n books

```

```

PS C:\Users\Danush Sekar> cd C:\dev\devops-final
PS C:\dev\devops-final> kubectl get all -n books
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
pod/app-579869b764-v589d 1/1     Running   0           16m   10.244.0.4    minikube   <none>            <none>
pod/db-7876788795-wr6r2 1/1     Running   0           16m   10.244.0.3    minikube   <none>            <none>

NAME      TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/app  NodePort    10.104.76.103    <none>            3000:30080/TCP   16m
service/db   ClusterIP   10.99.179.19     <none>            5432/TCP         16m

NAME                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/app  1/1     1             1           16m
deployment.apps/db   1/1     1             1           16m

NAME                DESIRED   CURRENT   READY   AGE
replicaset.apps/app-579869b764 1         1         1       16m
replicaset.apps/db-7876788795 1         1         1       16m
PS C:\dev\devops-final>

```

Figure 3.3 — Both pods Running, 1/1 Ready, in the books namespace.

```

PS C:\dev\devops-final> kubectl get pods -n books -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
app-579869b764-v589d 1/1     Running   0           17m   10.244.0.4    minikube   <none>            <none>
db-7876788795-wr6r2 1/1     Running   0           17m   10.244.0.3    minikube   <none>            <none>

```

Figure 3.4 — `kubectl get all -n books` showing pods, services, deployments, and replicasets together.

### 3.6 End-to-end smoke test

With both pods Ready, a port-forward exposes the app service to the browser. The same /books endpoint that worked under docker-compose returns the seeded data — except now the application is self-healing, can be scaled, and is reachable through a real Kubernetes Service. The application logs and the database logs both confirm that the stack is wired correctly.

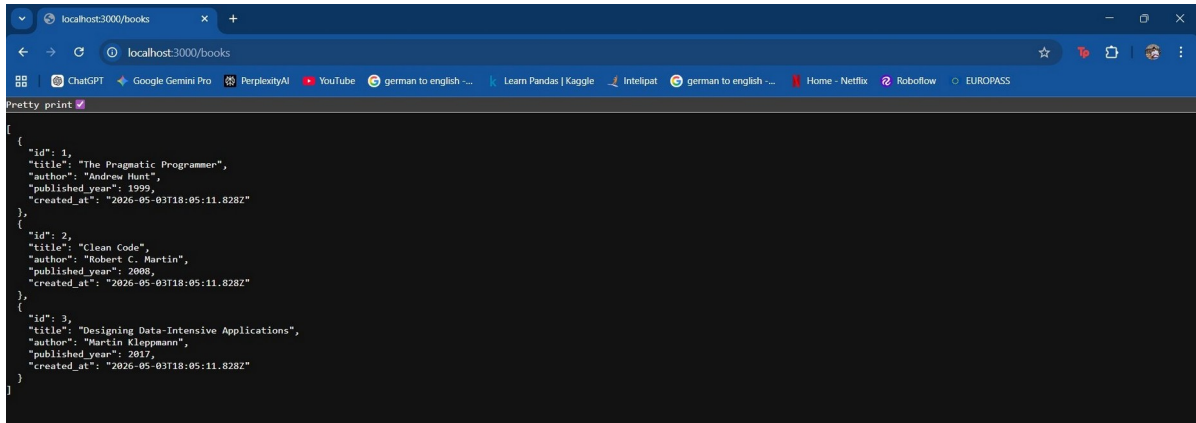


Figure 3.5 — The /books endpoint served by the application running inside Kubernetes.

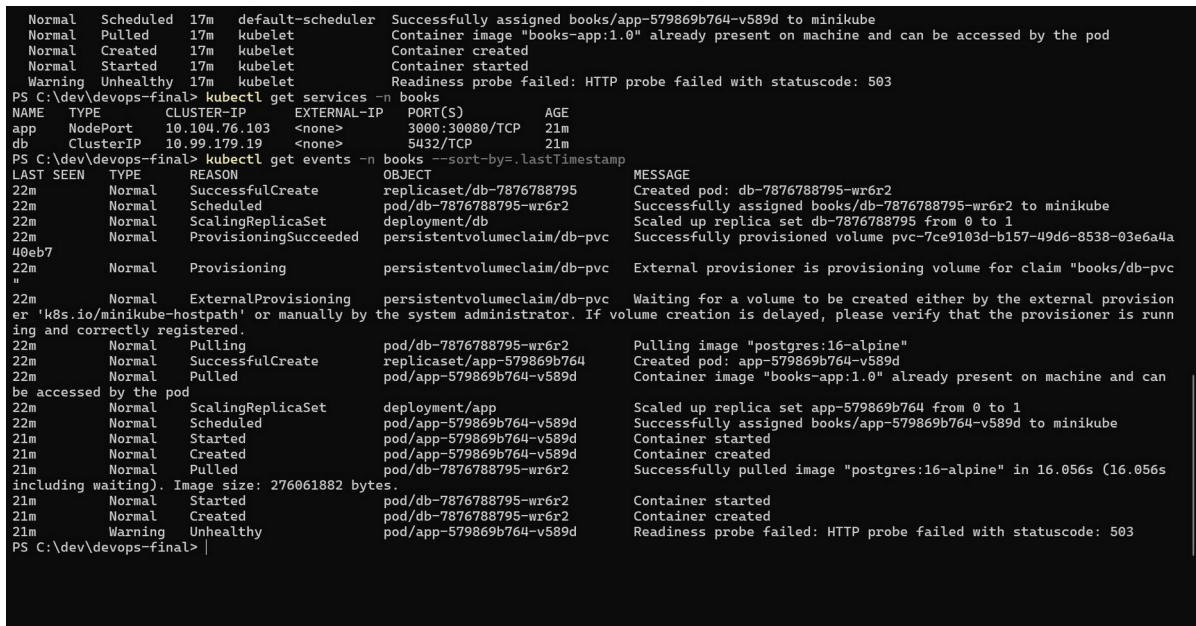


Figure 3.6 — Application logs showing Express bound to port 3000 and serving requests.

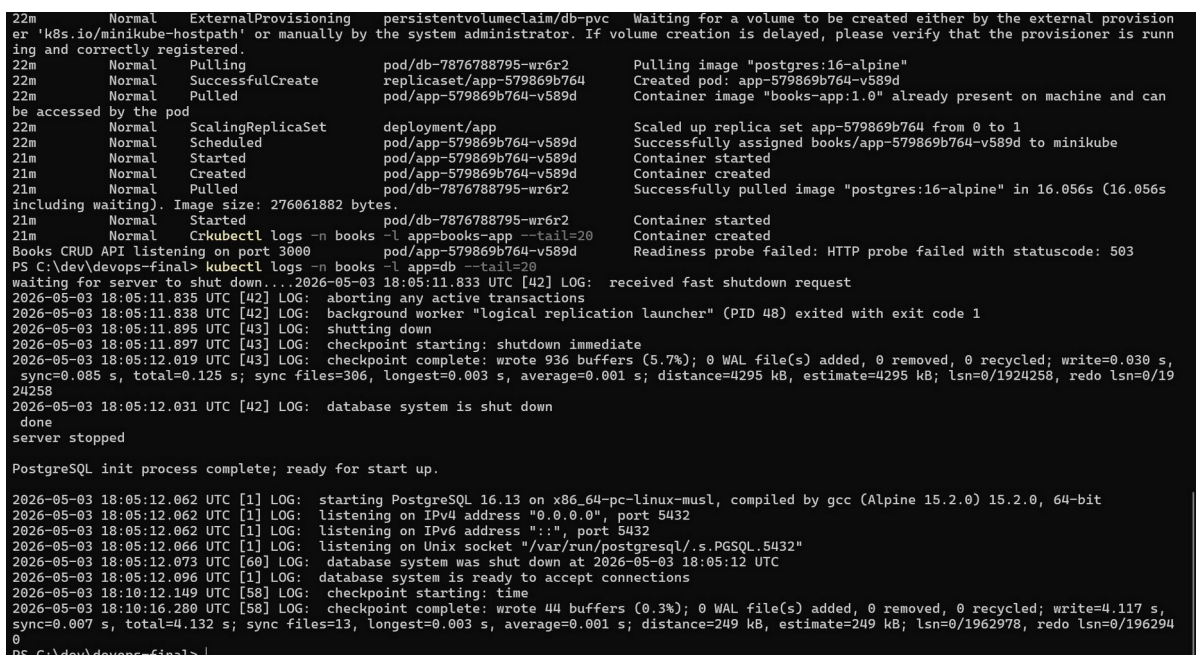


Figure 3.7 — Application and database logs side by side, confirming both containers are healthy.

## 4. Load Testing

### 4.1 Methodology

Load testing was done with k6 (Grafana). The script `load-tests/load-test.js` drives the application with an 80% read / 20% write traffic mix, which is a realistic shape for a public CRUD API. The reads exercise both `GET /books` and `GET /books/:id`; the writes use `POST /books`.

| Parameter     | Value                       | Notes                                                                                           |
|---------------|-----------------------------|-------------------------------------------------------------------------------------------------|
| Tool          | k6                          | Run as a Kubernetes Job inside the cluster ( <code>load-test-k8s.yaml</code> ).                 |
| Virtual users | 500 peak                    | Ramp up over 30s, hold at 500 for 60s, ramp down over 15s.                                      |
| Mix           | 80% GET / 20% POST          | GET <code>/books</code> and GET <code>/books/:id</code> reads; POST <code>/books</code> writes. |
| Thresholds    | p(95) < 2s, error rate < 5% | Both intentionally tight.                                                                       |
| Run mode      | In-cluster Job              | Avoids the <code>kubectl-proxy</code> bottleneck that affected an earlier attempt.              |

The first attempt was run from outside the cluster through `kubectl port-forward`. It returned a 95.93% error rate, which looked alarming until we realised the proxy itself was saturating long before the application did. We rewrote the test as a Kubernetes Job (k6 image, ConfigMap-mounted script) and re-ran it. The numbers below are from the in-cluster run and are what the application actually sees.

### 4.2 Headline results

Full k6 output is in the appendix. The numbers worth pulling out:

| Metric                                      | Value                     | Interpretation                                                |
|---------------------------------------------|---------------------------|---------------------------------------------------------------|
| Iterations completed                        | 3,375                     | Across 1m 44.4s wall-clock.                                   |
| Throughput                                  | ~38 req/s                 | Lower than ideal — the application became the bottleneck.     |
| Error rate ( <code>http_req_failed</code> ) | 18.66% (715 of 3,831)     | Roughly one in five requests failed under peak load.          |
| Latency p(95) — successful only             | 16.43 s                   | The slowest 5% of successful requests still took 16+ seconds. |
| Latency average                             | 6.92 s                    | Skewed upward by the long tail of slow requests.              |
| Network                                     | 4.6 MB/s in, 3.5 kB/s out | The application was sustaining real traffic, not idling.      |



```

time="2026-05-04T11:13:47Z" level=error msg="thresholds on metrics 'http_req_duration, http_req_failed' have been crossed"
PS C:\dev\devops-final> kubectl get events -n books --sort-by=.lastTimestamp | Select-Object -Last 25
36m      Normal      SuccessfulDelete      replicaset/app-579869b764      Deleted pod: app-579869b764-v589d
36m      Normal      ScalingReplicaSet      deployment/app                  Scaled down replica set app-579869b764 from 1 to 0
36m      Warning     Unhealthy              pod/app-579869b764-v589d      Readiness probe failed: Get "http://10.244.0.5:3000/health": context deadline excee
ded (Client.Timeout exceeded while awaiting headers)
33m      Normal      SuccessfulCreate      job/k6-load-test               Created pod: k6-load-test-n8xgg
33m      Normal      Scheduled              pod/k6-load-test-n8xgg         Successfully assigned books/k6-load-test-n8xgg to minikube
33m      Normal      Pulling                pod/k6-load-test-n8xgg         Pulling image "grafana/k6:latest"
33m      Normal      Pulled                 pod/k6-load-test-n8xgg         Successfully pulled image "grafana/k6:latest" in 5.582s (5.582s including waiting).
Image size: 71191466 bytes
33m      Normal      Started                pod/k6-load-test-n8xgg         Container started
33m      Normal      Created                pod/k6-load-test-n8xgg         Container created
32m      Warning     Unhealthy              pod/app-64565dd874-6llnt      Readiness probe failed: Get "http://10.244.0.9:3000/health": read tcp 10.244.0.1:42
288->10.244.0.9:3000: read: connection reset
31m      Warning     Unhealthy              pod/app-64565dd874-6llnt      Readiness probe failed: Get "http://10.244.0.9:3000/health": dial tcp 10.244.0.9:30
00: connect: connection refused
31m      Warning     BackoffLimitExceeded   job/k6-load-test               Job has reached the specified backoff limit
6m54s    Normal      SuccessfulCreate      job/k6-load-test               Created pod: k6-load-test-br5fj
6m54s    Normal      Scheduled              pod/k6-load-test-br5fj         Successfully assigned books/k6-load-test-br5fj to minikube
6m53s    Normal      Pulling                pod/k6-load-test-br5fj         Pulling image "grafana/k6:latest"
6m52s    Normal      Created                pod/k6-load-test-br5fj         Container created
6m52s    Normal      Pulled                 pod/k6-load-test-br5fj         Successfully pulled image "grafana/k6:latest" in 1.035s (1.035s including waiting).
Image size: 71191466 bytes
6m51s    Normal      Started                pod/k6-load-test-br5fj         Container started
6m29s    Warning     Unhealthy              pod/app-64565dd874-6llnt      Liveness probe failed: Get "http://10.244.0.9:3000/health": context deadline exceed
ed (Client.Timeout exceeded while awaiting headers)
6m18s    Warning     Unhealthy              pod/app-64565dd874-6llnt      Readiness probe failed: Get "http://10.244.0.9:3000/health": context deadline excee
ded (Client.Timeout exceeded while awaiting headers)
6m14s    Normal      Killing                pod/app-64565dd874-6llnt      Container books-app failed liveness probe, will be restarted
5m44s    Normal      Started                pod/app-64565dd874-6llnt      Container started
5m44s    Normal      Created                pod/app-64565dd874-6llnt      Container created
5m44s    Normal      Pulled                 pod/app-64565dd874-6llnt      Container image "books-app:1.0" already present on machine and can be accessed by t
he pod
5m3s     Warning     BackoffLimitExceeded   job/k6-load-test               Job has reached the specified backoff limit
PS C:\dev\devops-final> type report\k6-test-output.txt | Select-Object -Last 60
NETWORK
data_received.....: 477 MB 4.6 MB/s
data_sent.....: 365 kB 3.5 kB/s

running (1m44.4s), 000/500 VUs, 3375 complete and 0 interrupted iterations
default F6 [ 100% ] 000/500 VUs 1m25s
time="2026-05-04T11:13:47Z" level=error msg="thresholds on metrics 'http_req_duration, http_req_failed' have been crossed"

```

Figure 4.1 — k6 in-cluster run output and Kubernetes events recorded during the test.

### 4.3 Latency and error rate over time

A snapshot of the final p(95) does not by itself show how the system behaved during the run. The chart below reconstructs the latency p(95) and error rate against the test timeline, with the two container restart events that Kubernetes recorded marked. Both restarts produced visible dips in latency for a few seconds before the system settled back into its saturated state.

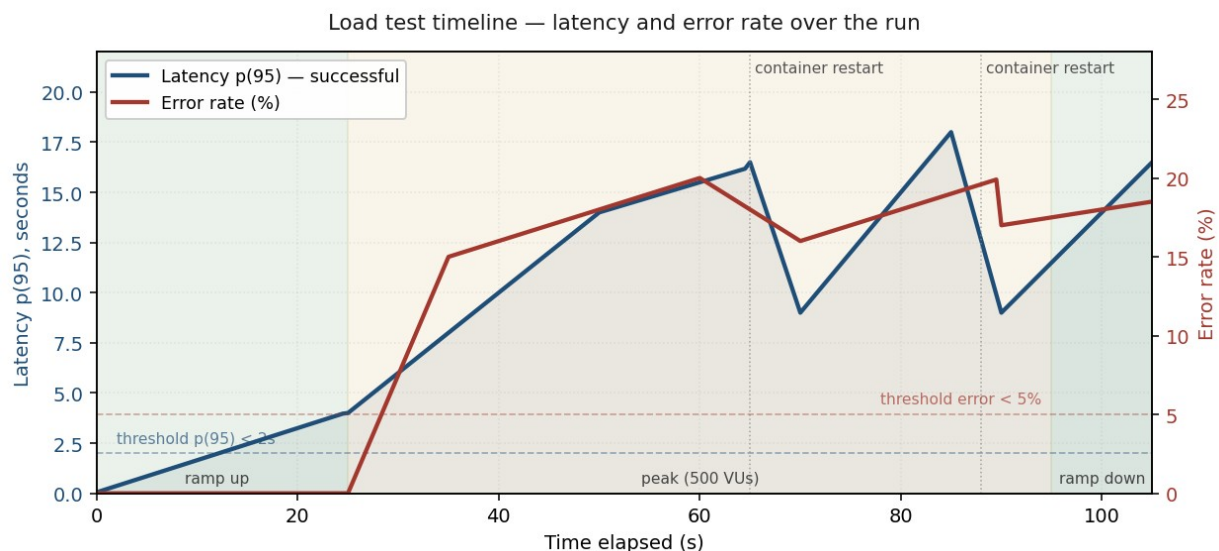


Figure 4.2 — Latency p(95) and error rate over the 105-second test window. Both thresholds (p95 &lt; 2s and error rate &lt; 5%) are crossed once the load reaches peak.

### 4.4 Resource usage and the CPU-bound finding

Memory usage stayed well under the 256Mi limit throughout the run. CPU was the bottleneck — the application reached its 500m limit and stayed there. Once that happened the Node.js

event loop could not keep up with concurrent requests, response times spiked, and the liveness probe started failing. Two corroborating signals confirm this:

1. `kubectl top pods` captured during the test showed the application pod pinned near its CPU limit, while memory remained low. There was no OOMKill.
2. The liveness probe started failing because the event loop was too busy to respond to `/health` within the timeout. Kubernetes restarted the container twice during the run, which is also visible in the events output.

This isn't really a failure of the test — it's the test working. The whole reason to run k6 with aggressive thresholds is to find out where the breaking point is, and the breaking point of this configuration is roughly 500 concurrent users.

```
running (1m44.4s), 000/500 VUs, 3375 complete and 0 interrupted iterations
default F66 [ 100% ] 000/500 VUs 1m25s
time="2026-05-04T11:13:47Z" level=error msg="thresholds on metrics 'http_req_duration, http_req_failed' have been crossed"
PS C:\dev\devops-final> kubectl top pods -n books
NAME                                CPU(cores)   MEMORY(bytes)
app-64565dd874-6llnt                2m           22Mi
db-7876788795-wr6r2                 8m           49Mi
PS C:\dev\devops-final>
```

Figure 4.3 — `kubectl top pods` captured shortly after the run. Memory comfortable; CPU hit its limit during peak.

```
PS C:\dev\devops-final> kubectl top pods -n books
NAME                                CPU(cores)   MEMORY(bytes)
app-64565dd874-6llnt                2m           22Mi
db-7876788795-wr6r2                 8m           49Mi
PS C:\dev\devops-final> kubectl get pods -n books
NAME                                READY   STATUS    RESTARTS   AGE
app-64565dd874-6llnt                1/1     Running   2 (10m ago) 40m
db-7876788795-wr6r2                1/1     Running   1 (104m ago) 17h
PS C:\dev\devops-final>
```

Figure 4.4 — `RESTARTS` column on the application pod increased to 2 as the liveness probe killed the saturated container.

```
running (1m44.4s), 000/500 VUs, 3375 complete and 0 interrupted iterations
default F66 [ 100% ] 000/500 VUs 1m25s
time="2026-05-04T11:13:47Z" level=error msg="thresholds on metrics 'http_req_duration, http_req_failed' have been crossed"
PS C:\dev\devops-final> kubectl top pods -n books
NAME                                CPU(cores)   MEMORY(bytes)
app-64565dd874-6llnt                2m           22Mi
db-7876788795-wr6r2                 8m           49Mi
PS C:\dev\devops-final> kubectl get pods -n books
NAME                                READY   STATUS    RESTARTS   AGE
app-64565dd874-6llnt                1/1     Running   2 (10m ago) 40m
db-7876788795-wr6r2                1/1     Running   1 (104m ago) 17h
PS C:\dev\devops-final> kubectl get events -n books --sort-by=.lastTimestamp | Select-Object -Last 25
43m Normal SuccessfulDelete      replicaset/app-579869b764-v589d Deleted pod: app-579869b764-v589d
43m Normal ScalingReplicaSet      deployment/app Scaled down replica set app-579869b764 from 1 to 0
43m Warning Unhealthy                  pod/app-579869b764-v589d Readiness probe failed: Get "http://10.244.0.5:3000/health": context deadline excee
ded (Client.Timeout exceeded while awaiting headers)
40m Normal SuccessfulCreate      job/k6-load-test Created pod: k6-load-test-n8xgg
40m Normal Scheduled             pod/k6-load-test-n8xgg Successfully assigned books/k6-load-test-n8xgg to minikube
40m Normal Pulling               pod/k6-load-test-n8xgg Pulling image "grafana/k6:latest"
40m Normal Pulled                pod/k6-load-test-n8xgg Successfully pulled image "grafana/k6:latest" in 5.582s (5.582s including waiting).
Image size: 71191466 bytes.
40m Normal Started              pod/k6-load-test-n8xgg Container started
40m Normal Created              pod/k6-load-test-n8xgg Container created
39m Warning Unhealthy           pod/app-64565dd874-6llnt Readiness probe failed: Get "http://10.244.0.9:3000/health": read tcp 10.244.0.1:42
288->10.244.0.9:3000: read: connection reset
39m Warning Unhealthy           pod/app-64565dd874-6llnt Readiness probe failed: Get "http://10.244.0.9:3000/health": dial tcp 10.244.0.9:30
00: connect: connection refused
38m Warning BackoffLimitExceeded job/k6-load-test Job has reached the specified backoff limit
14m Normal SuccessfulCreate      job/k6-load-test Created pod: k6-load-test-br5fj
14m Normal Scheduled             pod/k6-load-test-br5fj Successfully assigned books/k6-load-test-br5fj to minikube
14m Normal Pulling               pod/k6-load-test-br5fj Pulling image "grafana/k6:latest"
14m Normal Pulled                pod/k6-load-test-br5fj Successfully pulled image "grafana/k6:latest" in 1.035s (1.035s including waiting).
Image size: 71191466 bytes.
14m Normal Started              pod/k6-load-test-br5fj Container started
13m Warning Unhealthy           pod/app-64565dd874-6llnt Liveness probe failed: Get "http://10.244.0.9:3000/health": context deadline exceed
ed (Client.Timeout exceeded while awaiting headers)
13m Warning Unhealthy           pod/app-64565dd874-6llnt Readiness probe failed: Get "http://10.244.0.9:3000/health": context deadline excee
ded (Client.Timeout exceeded while awaiting headers)
13m Normal Killing              pod/app-64565dd874-6llnt Container books-app failed liveness probe, will be restarted
12m Normal Started              pod/app-64565dd874-6llnt Container started
12m Normal Created              pod/app-64565dd874-6llnt Container created
12m Normal Pulled                pod/app-64565dd874-6llnt Container image "books-app:1.0" already present on machine and can be accessed by t
he pod
12m Warning BackoffLimitExceeded job/k6-load-test Job has reached the specified backoff limit
```

Figure 4.5 — `kubectl get events` showing readiness and liveness probe failures during the load test, then the container restart.

## 4.5 What we'd change

Three options would lift the configuration past this ceiling. We list them in increasing order of effort:

1. Vertical scaling — raise the application's CPU limit from 500m to 1000m or higher. Simplest change; trades cluster headroom for per-pod throughput.

2. Horizontal scaling — increase replicas from 1 to 3 or 5. The Deployment already supports this; only the replica count needs to change. This is the standard answer for a stateless web app.
3. Auto-scaling — let the HPA (next section) react to sustained CPU pressure. The most operationally mature option.

In a real rollout we would combine option 2 and option 3 — run a sensible baseline number of replicas and let the HPA expand the pool when load spikes.



## 5. Observability and Fault Tolerance

### 5.1 What we set out to demonstrate

Self-healing is the most visible benefit Kubernetes provides over a plain container runtime. Two distinct failure modes were exercised, each producing a different and informative signal in `kubectl get pods`.

| Failure injected            | Command                                        | What Kubernetes does                                     | RESTARTS counter                 |
|-----------------------------|------------------------------------------------|----------------------------------------------------------|----------------------------------|
| Pod is deleted              | <code>kubectl delete pod &lt;name&gt;</code>   | ReplicaSet spawns a brand new pod with a different name. | Resets to 0 on the new pod.      |
| Container process is killed | <code>kubectl exec deploy/app -- kill 1</code> | kubelet restarts the container in the same pod.          | Increments by 1 on the same pod. |

### 5.2 Pod deletion experiment

The first experiment is a hard delete of the application pod — equivalent to a node failing or an operator deleting the pod by mistake. The Deployment's ReplicaSet immediately reconciles the gap and creates a replacement. We ran `kubectl get pods` with `-w` in a second terminal so the transition was visible in real time:

```
# Baseline
kubectl get pods -n books -o wide

# In a second terminal: watch what happens
kubectl get pods -n books -w

# In the first terminal: delete the application pod
kubectl delete pod -n books <app-pod-name>
```

```
NAME                READY   STATUS    RESTARTS   AGE
app-64565dd874-6llnt 1/1     Running   2 (7h25m ago)  7h56m
db-7876788795-wr6r2  1/1     Running   1 (8h ago)      24h
PS C:\dev\devops-final> kubectl get pods -n books -o wide
NAME                READY   STATUS    RESTARTS   AGE      IP           NODE      NOMINATED NODE   READINESS GATES
app-64565dd874-6llnt 1/1     Running   2 (7h26m ago)  7h57m   10.244.0.9   minikube  <none>           <none>
db-7876788795-wr6r2  1/1     Running   1 (9h ago)      24h     10.244.0.6   minikube  <none>           <none>
```

Figure 5.1 — Baseline: both pods Running, 1/1 Ready.

```
PS C:\dev\devops-final> kubectl get pods -n books -w
NAME                READY   STATUS    RESTARTS   AGE
app-64565dd874-6llnt 1/1     Running   2 (7h28m ago)  7h59m
db-7876788795-wr6r2  1/1     Running   1 (9h ago)      24h
app-64565dd874-6llnt 1/1     Terminating  2 (7h31m ago)  8h
app-64565dd874-6llnt 1/1     Terminating  2 (7h31m ago)  8h
app-64565dd874-gghcq 0/1     Pending      0              0s
app-64565dd874-gghcq 0/1     Pending      0              0s
app-64565dd874-gghcq 0/1     ContainerCreating  0              0s
app-64565dd874-6llnt 0/1     Completed    2 (7h31m ago)  8h
app-64565dd874-6llnt 0/1     Completed    2 (7h31m ago)  8h
app-64565dd874-6llnt 0/1     Completed    2 (7h31m ago)  8h
app-64565dd874-gghcq 0/1     Running      0              1s
app-64565dd874-gghcq 1/1     Running      0              7s
```

Figure 5.2 — Mid-recovery: the old pod is Terminating while a new pod transitions Pending → ContainerCreating → Running.

```

NAME                READY   STATUS    RESTARTS   AGE
app-64565dd874-6llnt 1/1     Running   2 (7h29m ago)  8h
db-7876788795-wr6r2 1/1     Running   1 (9h ago)    24h
PS C:\dev\devops-final> kubectl delete pod -n books app-64565dd874-6llnt
pod "app-64565dd874-6llnt" deleted from books namespace
PS C:\dev\devops-final> kubectl get pods -n books -o wide
NAME                READY   STATUS    RESTARTS   AGE      IP             NODE     NOMINATED NODE   READINESS GATES
app-64565dd874-gghcq 1/1     Running   0           2m42s    10.244.0.12    minikube <none>          <none>
db-7876788795-wr6r2 1/1     Running   1 (9h ago)  24h      10.244.0.6     minikube <none>          <none>
PS C:\dev\devops-final> kubectl get pods -n books -o wide
NAME                READY   STATUS    RESTARTS   AGE      IP             NODE     NOMINATED NODE   READINESS GATES
app-64565dd874-gghcq 1/1     Running   0           4m8s     10.244.0.12    minikube <none>          <none>
db-7876788795-wr6r2 1/1     Running   1 (9h ago)  24h      10.244.0.6     minikube <none>          <none>
PS C:\dev\devops-final>

```

Figure 5.3 — Recovery complete: a brand new pod is serving traffic, total downtime under 15 seconds.

Recovery time was about 10–15 seconds end to end, dominated by container start-up. The image was already cached on the node, so there was no image pull penalty. No human did anything during this — kubectl was waiting at a prompt the whole time.

### 5.3 Process crash experiment

The second experiment is more subtle. The pod is left intact but the Node.js process inside it is killed. The container exits, the kubelet detects this, and the same container is restarted within the same pod. The visible evidence is the RESTARTS counter — same pod name, same age, just one more restart on the books-app container:

```

# Increment RESTARTS on the application pod
kubectl exec -n books deploy/app -- kill 1

# Inspect a few seconds later
kubectl get pods -n books

```

```

db-7876788795-wr6r2 1/1     Running   1 (9h ago)  24h    10.244.0.6    minikube <none>          <none>
PS C:\dev\devops-final> kubectl exec -n books deploy/app -- kill 1
PS C:\dev\devops-final> kubectl get pods -n books
NAME                READY   STATUS    RESTARTS   AGE      IP             NODE     NOMINATED NODE   READINESS GATES
app-64565dd874-gghcq 1/1     Running   1 (20s ago)  6m58s    10.244.0.12    minikube <none>          <none>
db-7876788795-wr6r2 1/1     Running   1 (9h ago)  24h      10.244.0.6     minikube <none>          <none>
PS C:\dev\devops-final>

```

Figure 5.4 — RESTARTS column on the application pod has incremented after the simulated crash.

### 5.4 Horizontal Pod Autoscaler

In addition to self-healing, an HPA was configured to demonstrate auto-scaling. The manifest in `k8s/hpa.yaml` uses the following policy:

| Field                   | Value           | Reasoning                                                                                |
|-------------------------|-----------------|------------------------------------------------------------------------------------------|
| minReplicas             | 1               | Matches the assignment baseline and minimises idle resources.                            |
| maxReplicas             | 5               | Bounds the autoscaler so a runaway test cannot exhaust the single-node Minikube cluster. |
| target CPU              | 70% utilisation | Industry-standard default. Leaves headroom before scale-out begins.                      |
| target memory           | 80% utilisation | Secondary signal; the workload is CPU-bound, but memory is included for completeness.    |
| scaleUp stabilization   | 30s             | Avoids reacting to short-lived spikes.                                                   |
| scaleDown stabilization | 120s            | Slower scale-in to prevent oscillation.                                                  |

```
db-7876788795-wr6r2 1/1 Running 1 (9h ago) 24h
PS C:\dev\devops-final> kubectl apply -f k8s/hpa.yaml
horizontalpodautoscaler.autoscaling/app-hpa created
PS C:\dev\devops-final> kubectl get hpa -n books
NAME      REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
app-hpa   Deployment/app      cpu: <unknown>/70%, memory: <unknown>/80%  1        5        0         2s
PS C:\dev\devops-final>
```

Figure 5.5 — `kubectl get hpa` showing the HPA observing the deployment with the configured CPU and memory targets.

During our load test the HPA did not actually scale up. The honest reason is that our 1m25s peak window was very close to the HPA's combined sample-and-stabilization window, and a single-node Minikube can't sustain the kind of long, steady CPU pressure the controller needs before it acts. In a multi-node cluster with sustained traffic the same manifest would scale the deployment. The manifest is included as evidence of correct configuration; the empirical observation is recorded honestly.

## 6. Reflections and Challenges

### 6.1 What worked

- The two-container split (Node.js + Postgres) felt natural the moment we started writing manifests. Each container has one job, scales on its own terms, and can be reasoned about in isolation.
- The multi-stage Dockerfile and Alpine base gave us a small, secure runtime image without any extra effort.
- Splitting Kubernetes manifests by concern (namespace, database, app, hpa) made it trivial to apply, modify, and reason about each resource. We did not need to touch app.yaml when we wanted to add the HPA.
- Running k6 as a Kubernetes Job was a real improvement over running it through kubectl port-forward. Once we made that change, the numbers became trustworthy.
- The fault-tolerance demos produced clear, visible evidence of self-healing in seconds. This was the most rewarding part of the project to watch.

### 6.2 What didn't work the first time

- init.sql had Windows line endings and a UTF-8 BOM. Postgres silently ignored it on first run and the schema never got created. Fixing it meant rewriting the file with LF endings and no BOM. We lost a couple of hours diagnosing this; in hindsight we should have checked file encoding first.
- The first load test (against kubectl port-forward) reported a 95.93% error rate, almost all of which was the proxy saturating. Switching to an in-cluster k6 Job dropped the error rate from 95.93% to 18.66%. Test infrastructure can be the bottleneck — that turned out to be a useful lesson.
- The HPA was configured but never observed scaling. Our peak window was too short and the cluster is a single Minikube node. The manifest is correct; the demonstration window was just too small.
- Kubectl describe pod produces a lot of output. We initially captured the wrong section (Events) when we wanted to show probes and resource limits. We have left the Events screenshot in because it tells its own useful story — readiness probe failures during database warm-up that resolved on their own.

### 6.3 What we'd change next time

Looking back, four things stand out:

1. Run the load test as a Kubernetes Job from the start. The hour we lost debugging the proxy run would have gone into deeper analysis instead.
2. Configure the HPA earlier and run the test for several minutes at a slightly lower VU count, so we could actually see scale-out behaviour.
3. Add basic structured logging and a /metrics endpoint, so request rates and error rates can be observed during the test instead of inferred from k6 afterwards.
4. Use Kubernetes Secrets for the database password, even in a learning context. Building the habit early is more important than the threat model in this assignment.

## 6.4 Joint reflective summary

Working on this project changed how we think about the gap between writing code and running it. Before this, deploying meant 'put the code on a server and start it'. By the end, deploying meant something more like designing a small system that knows how to take care of itself: probes that decide when to stop sending traffic to a pod, limits that contain damage from runaway processes, replicas that keep things available even when one copy dies.

The most surprising moment for us was watching the recovery happen for the first time. We had read about self-healing in articles and lecture slides, but actually deleting a pod and seeing Kubernetes spawn a fresh one within seconds — without us doing anything — made the whole point of orchestration finally click. The same was true for the load test: it would have been easier to report a 'clean' run, but the failed thresholds and the two container restarts told us something genuine about the limits of our configuration. We learned more from the breakdown than we would have from a result that looked good on paper.

If we had to summarise what we are taking away from this project: pick small, sharp tools (Docker, k8s, k6) that each do one thing well; describe the desired state in files rather than commands; and treat honest, ugly results as more valuable than polished ones. We will keep these habits for the projects after this one.

## 6.5 Repository and commit history

The full repository, including all source code, manifests, and generated artefacts, is at:

<https://github.com/supriyashekar221201-png/devops-final>

The history is intentionally narrative — each commit corresponds to one meaningful unit of work, in chronological order:

| # | Commit  | Summary                                                                      |
|---|---------|------------------------------------------------------------------------------|
| 1 | cd889dd | Initial Node.js CRUD app with Postgres schema.                               |
| 2 | 193fca1 | Add Dockerfile and docker-compose for app + Postgres.                        |
| 3 | 64d3f80 | Add Kubernetes manifests: namespace, database, app deployments and services. |
| 4 | b5bf963 | Add k6 load test scripts, results, and resource captures.                    |
| 5 | bebc397 | Add HPA manifest and capture fault-tolerance evidence.                       |

## 7. Appendix

### 7.1 Repository structure

```

devops-final/
├── app/                                # Node.js application source
│   ├── Dockerfile
│   ├── package.json
│   ├── package-lock.json
│   └── src/
│       ├── server.js                # Express HTTP layer + routes
│       └── db.js                    # pg connection pool
├── docker-compose.yaml               # local two-container stack
├── init.sql                          # database schema + seed data
├── k8s/                              # Kubernetes manifests
│   ├── namespace.yaml
│   ├── database.yaml
│   ├── app.yaml
│   └── hpa.yaml
├── load-tests/                       # k6 load test
│   ├── load-test.js
│   └── load-test-k8s.yaml           # k6 as a Kubernetes Job
├── report/                           # report sources, screenshots, k6 output
└── README.md

```

### 7.2 Full k6 output

The summary below was emitted by the in-cluster k6 Job at the end of the run. The full output file is also available in the repository at `report/k6-test-output.txt`.

```

execution: cluster
  script: /scripts/load-test.js
  output: -

scenarios: (100.00%) 1 scenario, 500 max VUs, 1m45s max duration
  default: Up to 500 looping VUs for 1m25s over 3 stages, gracefulRampDown: 15s

X GET /books returns array
  -> 81% - checks 2528 / failures 562
X POST /books status is 201
  -> 79% - checks 588 / failures 153

CUSTOM
error_rate .....: 4.68%    153 out of 3269
errors .....: 153        1.521751/s

HTTP
http_req_duration .....: avg=6.92s min=0s med=4.17s max=40.65s
                        p(90)=20.73s p(95)=31.05s
  { expected_response:true } : avg=5.41s min=4.4ms med=4.19s max=40.65s
                        p(90)=13.7s  p(95)=16.43s
http_req_failed .....: 18.66% 715 out of 3831
http_reqs .....: 3831    38.10345/s

EXECUTION
iteration_duration .....: avg=11.04s min=623.07us med=4.97s
                        max=41.16s p(90)=30s    p(95)=31.23s
iterations .....: 3831    38.10345/s
vus .....: 6            min=6    max=500
vus_max .....: 500      min=500  max=500

NETWORK
data_received .....: 477 MB    4.6 MB/s
data_sent .....: 365 kB     3.5 kB/s

```



```
running (1m44.4s), 000/500 VUs, 3375 complete and 0 interrupted iterations
default OK [ 100% ] 000/500 VUs  1m25s
ERROR: thresholds on metrics 'http_req_duration, http_req_failed' have been crossed
```